

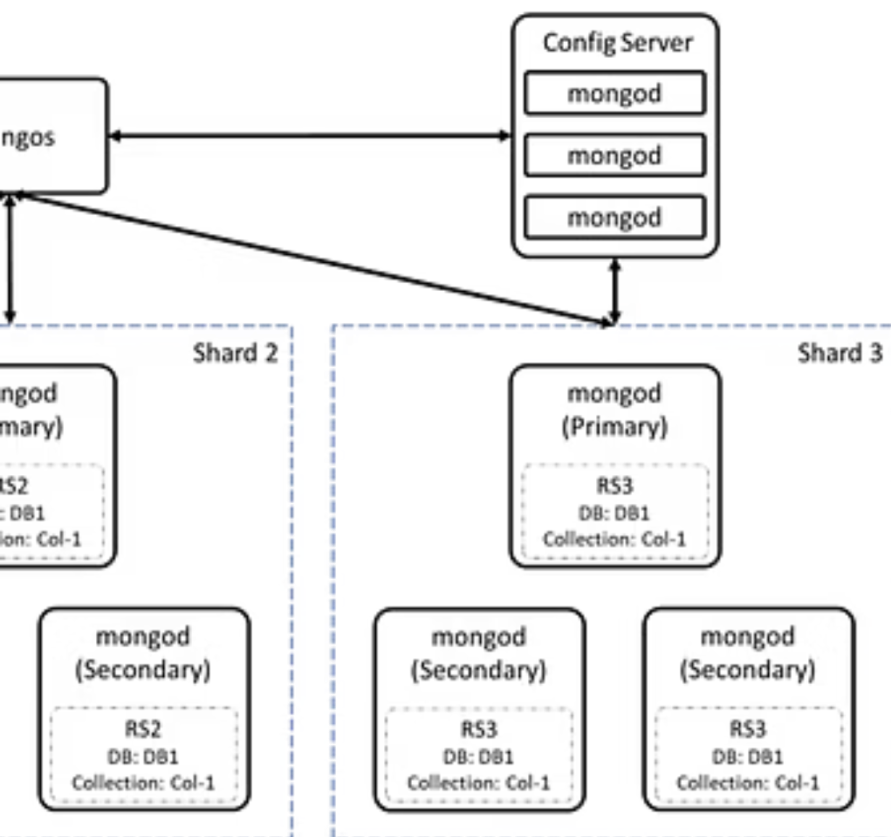
Bancos de Dados

Aula - 04

2026

SENAI - CRTI

Introduction To mong



Tipos de Dados no MongoDB

Unidade Curricular: Bancos de Dados

Nesta aula, vamos explorar os principais tipos de dados utilizados no MongoDB, entender como os documentos são estruturados e reconhecer a importância da tipagem em bancos de dados NoSQL.

Um Problema Real no Desenvolvimento

O Cenário

Uma aplicação precisa armazenar dados variados de usuários: **nomes**, **datas de cadastro**, **listas de interesses** e **endereços**. O desenvolvedor iniciante começa a inserir dados no MongoDB, mas não compreende quais tipos de dados está utilizando.

Erros surgem: comparações incorretas, filtros que não funcionam, dados inconsistentes.



Problema identificado: Sem entender os tipos de dados do MongoDB, o desenvolvedor não consegue modelar, consultar ou validar as informações corretamente.

Pergunta para reflexão: Você saberia dizer qual tipo de dado representa cada campo em um documento MongoDB?

O Que é um Documento no MongoDB?

Estrutura JSON / BSON

No MongoDB, os dados são armazenados em **documentos** — estruturas semelhantes ao formato JSON (JavaScript Object Notation). Internamente, o MongoDB usa **BSON** (Binary JSON), que estende o JSON com tipos adicionais como Date e ObjectId.

- Cada documento é um conjunto de pares **chave: valor**
- Os documentos ficam dentro de **coleções** (collections)
- Não exige esquema fixo (schema-less)

Exemplo de Documento

```
{
  nome: "João",
  idade: 25,
  ativo: true,
  interesses: [
    "tecnologia",
    "esportes"
  ],
  endereco: {
    cidade: "Rio",
    estado: "RJ"
  },
  cadastro: new Date(),
  _id: ObjectId("...")
}
```



String e Number

String

Representa **texto** — qualquer sequência de caracteres entre aspas.

```
{ nome: "João",  
  cidade: "Rio de Janeiro",  
  email: "joao@email.com" }
```

Usado para: nomes, e-mails, descrições, endereços.

Number

Representa valores **numéricos** — inteiros ou decimais.

```
{ idade: 25,  
  salario: 3500.50,  
  pontuacao: 98 }
```

Usado para: idades, preços, quantidades, pontuações.



Atenção: No MongoDB, "25" (String) é diferente de 25 (Number). Ao filtrar, o tipo importa!

Boolean e Null

Boolean — Verdadeiro ou Falso

Armazena apenas dois valores possíveis: **true** ou **false**. Ideal para representar estados, permissões e flags em sistemas.

```
{ ativo: true,  
  emailVerificado: false,  
  admin: false }
```

✔ Usado para: status de conta, permissões de acesso, preferências do usuário.

Null — Ausência de Valor

Representa a **ausência intencional** de um valor. Diferente de zero ou string vazia — significa que o campo existe, mas não tem dado.

```
{ apelido: null,  
  foto: null,  
  deletadoEm: null }
```

📌 Usado para: campos opcionais não preenchidos, datas ainda não definidas, informações pendentes.

Array — Lista de Valores

O que é um Array?

Um **Array** é uma lista ordenada de valores dentro de colchetes `[ ]`. Pode conter qualquer tipo de dado — strings, números, objetos ou até outros arrays.

```
{ interesses: [
  "tecnologia",
  "esportes",
  "música"
],
notas: [8.5, 9.0, 7.8],
tags: ["admin", "ativo"]
}
```

Por que usar Arrays?

- Armazenar **múltiplos valores** em um único campo
- Evitar criar documentos duplicados para cada item
- MongoDB suporta **consultas diretas** dentro de arrays

```
{
  "_id": 1,
  "items": [ "Apple", "Banana", "Orange", "Apple" ]
},
{
  "_id": 2,
  "items": [ "Banana", "Grapes", "Apple", "Mango" ]
}
```

Object — Documento Dentro de Documento

Documentos Aninhados (Embedded)

O tipo **Object** permite inserir um documento completo como valor de um campo. Isso é chamado de **documento aninhado** ou *embedded document* — um dos recursos mais poderosos do MongoDB.

```
{ nome: "João",  
  endereco: {  
    rua: "Av. Brasil",  
    numero: 100,  
    cidade: "Rio",  
    estado: "RJ",  
    cep: "20000-000"  
  }  
}
```

Vantagens

Dados Relacionados Juntos

Informações de endereço ficam agrupadas com o usuário, sem necessidade de tabela separada.

Consultas Detalhadas

É possível filtrar por `endereco.cidade` usando notação de ponto.

Estrutura Flexível

Cada documento pode ter uma estrutura de objeto diferente, conforme a necessidade.

Date — Trabalhando com Datas

O Tipo Date no MongoDB

O tipo **Date** armazena datas e horários no formato **ISODate**, internamente como milissegundos desde 1º de janeiro de 1970 (Unix Timestamp). É um tipo nativo do BSON.

```
{ nome: "João",  
  dataCadastro:  
    new Date("2024-01-15"),  
  ultimoAcesso:  
    new Date(),  
  dataNascimento:  
    ISODate("1999-05-20")  
}
```

Usos Comuns em Sistemas



Registros de Criação

Data de cadastro de usuários e criação de documentos.



Logs e Auditoria

Registrar quando uma ação foi realizada no sistema.



Filtros por Período

Consultar documentos criados em um intervalo de datas.

ObjectId — O Identificador Único

O que é o ObjectId?

O **ObjectId** é um tipo especial do BSON usado como **identificador único** de cada documento no MongoDB. Por padrão, o campo `_id` de todo documento recebe um ObjectId gerado automaticamente.

```
{ _id: ObjectId(
  "64b1f23a9c4e5d001a2b3c4d"
),
  nome: "João",
  idade: 25
}
```

O valor tem **12 bytes** e inclui: 4 bytes de timestamp, 5 bytes de identificador da máquina e processo e 3 bytes de contador incremental.

Por que é Importante?

Unicidade

Garante que nenhum documento tenha o mesmo `_id`.

Referências

Usado para referenciar documentos entre coleções diferentes.

Eficiência

Gerado automaticamente, sem necessidade de sequência manual.

Resumo: Tipos de Dados no MongoDB

1

String

```
"João", "Rio"
```

Texto e caracteres

2

Number

```
25, 3500.50
```

Inteiros e decimais

3

Boolean

```
true / false
```

Estados e flags

4

Array

```
["tec", "esp"]
```

Listas de valores

5

Object

```
{ cidade: "Rio" }
```

Documentos aninhados

6

Date

```
new Date()
```

Datas e horários

7

Null

```
null
```

Ausência de valor

8

ObjectId

```
ObjectId("...")
```

Identificador único



Por que tipagem importa? Usar o tipo correto garante **consultas precisas, validação de dados, melhor performance** e um sistema mais **confiável e previsível**.

